

Appendix G - Changes Compared to Assignment 1

- Course: Event-driven and Process-oriented Architectures (EDPO), FS2026, University of St.Gallen
- Group 4
 - Evan Martino
 - Marco Birchler
 - Roman Babukh

Further Information

- Assignment 1 GitHub Release:
https://github.com/chechmek/EDPO_FS26_Project/releases/tag/assignment-1
 - Assignment 2 GitHub Release:
https://github.com/chechmek/EDPO_FS26_Project/releases/tag/assignment-2
-

Overview

All Assignment 1 concepts, architecture decisions, and documentation were left unchanged after the Assignment 1 submission. No retroactive edits were made to Assignment 1 documents, BPMN models, ADRs, or service logic as such. The focus after the submission was exclusively on satisfying Assignment 2 requirements.

The only changes made to the pre-existing codebase were minimal additions to the Kafka event payloads of two services so that the new stream processors could consume them. Everything else that changed was strictly additive: new services, new processors, new scripts, and extended infrastructure configuration.

Changes to existing Services

`services/verification-service/app.py`

The `verification-notification` Kafka event payload was extended with three new fields required by the stream processors:

Field added	Purpose
<code>eventTime</code>	ISO-8601 UTC timestamp used for event-time processing and window placement
<code>contentId</code>	Stable SHA-256-derived identifier for a content URL, used as the stream-processing key
<code>status</code>	Duplicates the existing <code>type</code> field in a normalised form expected by the processors

A small helper function `_content_id_for_url()` was added to derive a deterministic `contentId` from the content URL. No existing fields were removed or renamed and no BPMN workers were changed.

`services/reporting-service/app.py`

The `report-notification`, `post-deleted`, and `objection-approved` Kafka event payloads were extended with the same set of fields:

Field added	Purpose
<code>eventTime</code>	ISO-8601 UTC timestamp for event-time processing
<code>contentId</code>	Passed through from the <code>postId</code> , used as the stream-processing join key
<code>status</code>	Normalised status string matching the processor's event type expectations

A small helper function `_utc_now_iso()` was added. No existing fields, workers, or BPMN tasks were changed.

`docker-compose.yml`

Three new services were added:

Service	Description
<code>sla-monitor</code>	Python stream processor, exposed on port <code>8005</code>
<code>content-ledger</code>	Java / Kafka Streams processor, exposed on port <code>8085</code>
<code>ui</code>	Lightweight Flask dashboard aggregating both processor APIs, exposed on port <code>8086</code>

No existing service definitions were modified.

Additions for Assignment 2

Everything listed below was newly created and did not exist at the Assignment 1 release:

Path	What it is
<code>processors/sla-monitor/</code>	Full Python SLA Monitor processor with Flask IQ API
<code>processors/content-ledger/</code>	Full Java / Kafka Streams Content Decision Ledger processor
<code>ui/</code>	Flask-based web UI proxying both processor APIs
<code>schemas/content-ledger/</code>	Avro schema definitions for <code>ContentEvent</code> and <code>ContentDecisionState</code>
<code>scripts/</code>	Demo and load-generation scripts for testing the processors
<code>misc/content-ledger-postman-collection.json</code>	Postman collection for the Content Ledger REST API
<code>docs/(...-assignment-2)</code>	Documentation reagrding Assignment 2
<code>README.md</code>	Updated to cover the full Assignment 2 stack and setup instructions

